A Comprehensive Theoretical and Experimental Comparison of Dijkstra's Algorithm and Duan et al.'s SSSP Algorithms

Anmol, Tanmay Arora, Adhyyan, Pahul Veer Singh Gogna
Department of Computer Science & Engineering
Chitkara University
Punjab, India
anmol0115.be23@chitkara.edu.in, tanmay1305.be23@chitkara.edu.in,
adhyyan0180.be23@chitkara.edu.in, pahul0508.be23@chitkara.edu.in

***Abstract*. Dijkstra's algorithm has been widely used as the standard practical method for the single source shortest path (SSSP) problem for nearly six decades, but its dominant position has recently been challenged. In 2025 the first deterministic algorithm for directed graphs with nonnegative real weights that is asymptotically faster on sparse graphs was introduced by Duan, Mao, Mao, Shu and Yin, and in 2026 this approach was further refined by Duan, Mao, Shu and Yin, who presented an improved algorithm with a better theoretical bound. Although both algorithms are primarily theoretical, their potential impact is significant. In this study all three algorithms were comprehensively compared under identical conditions: a component-by-component comparison of the two new algorithms was conducted, both were implemented directly from their published pseudocode, and they were validated on 200 000 and 30 000 random graphs. Running time and machine-independent counts of comparisons, additions and edge scans were carefully evaluated on eleven real world graphs and four synthetic families, some containing up to four million vertices. Dijkstra's algorithm, particularly when implemented with a binary heap, consistently achieved the best performance across all instances; on the five major road networks it was 3.5 to 8.0 times faster than the other algorithms. On random and preferential-attachment graphs, where a wide search frontier is typically encountered, the earlier of the two new algorithms required fewer comparisons than Dijkstra's algorithm on large graphs, demonstrating its asymptotic advantage under the theoretical cost model; nevertheless, at four million vertices it was still approximately 1.5 times slower than Dijkstra's algorithm. The later algorithm attempts to reduce much of the pivot-search overhead of the earlier version, but surprisingly it was faster than its predecessor on only one instance. Instrumented runs also detected a previously known stale-key error of the earlier algorithm, which, alarmingly, also appears in the published pseudocode of the later one, and its base case can return a boundary that violates its own invariant; fortunately, neither issue affected any computed distance.**

***Keywords*. single source shortest paths, Dijkstra's algorithm, bounded multi-source shortest path, sorting barrier, experimental algorithmics, comparison-addition model**

I. Introduction

The single source shortest path (SSSP) problem is a fundamental problem in graph theory: the shortest path distances from a designated source vertex s to all other vertices of a directed graph $G = (V, E)$ are required, where the graph has $n = |V|$ vertices and $m = |E|$ directed arcs with nonnegative real weights on every arc. A solution was first provided by Dijkstra's algorithm [1]: when a Fibonacci heap [2] or a relaxed heap [3] is used to manage the frontier, a running time of $O(m + n \log n)$ is achieved, in the comparison-addition model in which weights can only be added and compared. A key characteristic of Dijkstra's algorithm is that the vertices are settled in order of increasing distance from the source, so they are effectively sorted as a side product, and for a long time the $n \log n$ term was regarded as an unavoidable barrier for sparse graphs. This intuition was later clarified by Haeupler et al. [4]: an appropriate heap implementation of Dijkstra's algorithm was shown to be universally optimal among the algorithms that must produce the vertices in distance order. This distinction is important, because an algorithm that returns only the distances, without producing the vertices in sorted order, is not subject to the same limitation.

This limitation was addressed by Duan, Mao, Mao, Shu and Yin [5]. A deterministic algorithm with a running time of $O(m \log^{2/3} n)$ was introduced, the first to surpass the $O(m + n \log n)$ barrier on sparse directed graphs with real weights. Its central innovation is a recursive technique, the bounded multi-source shortest path (BMSSP) procedure, in which the whole frontier is not kept sorted: instead, the frontier is reduced to a small collection of pivots through a specialized subroutine, and a block-based structure distributes the smallest frontier elements without ordering them internally. In 2026 this approach was significantly developed by Duan, Mao, Shu and Yin [6]: an improved bound of $O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$, which simplifies to $O(m\sqrt{\log n \log\log n})$ for sparse graphs, was established by replacing the Bellman Ford style pivot search with local Dijkstra searches and a forest partition and by simplifying the data structure. Since neither paper assigns a name to its algorithm, the algorithm of Duan, Mao, Mao, Shu and Yin [5] is referred to here as BMSSP-25 and the improved algorithm of Duan, Mao, Shu and Yin [6] as BMSSP-26, where the suffix indicates the year of publication.

Neither paper provides a detailed discussion of practical implementation. Nevertheless, an experimental evaluation is

valuable for three main reasons. First, the asymptotic improvement is relatively modest: $\log^{1/3} n$ is 2.7 for $n = 2^{20}$ and increases to only 4.0 for $n = 2^{64}$, so a theoretical advantage need not translate into better practical performance unless the constant factors are carefully controlled. Second, potentially large constant factors are involved in both algorithms, since the arcs must be scanned several times and highly specialized, custom-built data structures must be maintained. Third, the assumptions underlying the analyses are not straightforward to satisfy in practice: both rely on a degree-bounded graph obtained through a transformation, and both require distinct path lengths, which are generally ensured through tie-breaking. BMSSP-25 has already been evaluated in earlier studies [7]–[9]; to the best of our knowledge, however, no peer reviewed empirical evaluation of BMSSP-26 has yet been published, and the algorithms have not previously been evaluated in the same model in which their running time bounds were originally established.

The following contributions are presented in this paper.

- A detailed, component-by-component comparison of Dijkstra's algorithm, BMSSP-25 and BMSSP-26 is provided, with particular attention to the parameter values the algorithms use at practical input sizes (Section III).
- Independent C++ implementations of both BMSSP algorithms were developed directly from the published pseudocode, and randomized differential testing was conducted to validate their correctness (Sections IV and VI-A).
- Two issues in the published pseudocode of BMSSP-26 were identified during validation: completed vertices can remain stored in the caller's data structure, as in BMSSP-25, and the base case can return a boundary greater than a value it leaves behind (Section VI-A).
- Extensive measurements of running time, comparisons, additions and edge scans were conducted on five real road networks, six further graphs from the SNAP collection and four synthetic families, with sweeps over the parameters k and t and over the degree reduction, density and weight range (Section VI).

II. Related Work

A. Classical results

An $O(m + n \log n)$ running time bound for Dijkstra's algorithm was established by Fredman and Tarjan [2] through Fibonacci heaps, and this result was complemented by the relaxed heaps of Driscoll et al. [3]. Experimental research on shortest path algorithms has a long tradition [10], and in practice Dijkstra's algorithm is generally equipped with a binary heap, as in [7]. Faster algorithms have been developed, although they generally fall outside the scope of the present study: Thorup's linear-time algorithm [11] is restricted to undirected graphs with integer weights in the word RAM model, and the algorithm of Pettie and Ramachandran [12] is also designed for undirected graphs. For undirected graphs with real weights, a randomized $O(m\sqrt{\log n \log\log n})$ algorithm was introduced by Duan et al. [13] and later derandomized by Yan [14]; a further improvement for undirected graphs appeared while the present study was being prepared [15].

B. Experimental studies of BMSSP-25

BMSSP-25 was implemented in C++ by Castro, Clementino and de Freitas [7] in three variants (worst case, constant degree, expected time). The first variant was compared with binary-heap Dijkstra on sparse random graphs containing up to 2^{25} vertices, as well as on grids and United States road networks. Dijkstra's algorithm was observed to be three to four times faster on large instances and up to 27.5 times faster on small grids. Through curve fitting, a potential crossover point was estimated to lie far beyond any practical size, and the constant-degree transformation was concluded to be detrimental to practical performance. A critical error in the original analysis was also identified: a vertex already completed within a recursive call could remain in the data structure with an outdated key, so the sets returned by successive calls were not necessarily disjoint; this was resolved by removing each vertex immediately upon completion, and both the problem and the correction were confirmed by the original authors [7]. Their implementation was made publicly available [16]. A Rust implementation was evaluated by Valko, Paranjpe and Marx Gómez [8] on snapshots of the Lightning Network, and Dijkstra's algorithm was likewise found to be faster. A Python package was provided by Makowski et al. [9], through which measurements on grid and geospatial networks were also reported.

Several public repositories have also claimed to implement BMSSP-25, with some reporting better performance than Dijkstra's algorithm [17], [18]. Closer examination of their source code, however, reveals that these results often concern variants that differ from the published algorithm. For example, the optimized solver of [17] replaces the recursion over levels, the pivot search and the block structure with a bucket queue of fixed bucket width, and it was evaluated on queries from one source to one target in unit-weight graphs, where distance-based bucketing achieves linear-time performance outside the comparison-addition model; the same repository reports that its solver which follows the paper more closely is two to four times slower than Dijkstra. Similarly, the comparison in [18] concerns queries from one source to one target on unit-weight graphs against a standard library routine. These results are therefore not considered direct measurements of the algorithm of [5], but the graphs used in these studies were included in our experiments (Section V).

The studies above primarily report the wall clock running time of BMSSP-25 (with memory also reported in [7]). Several important questions remain open, however: how BMSSP-26 performs in practice, how much of the observed gap is attributable to additional algorithmic work rather than the cost of individual operations, and how performance depends on graph structures beyond random graphs, grids and road networks.

III. The Three Algorithms

A. A common view

All three algorithms maintain tentative labels $\widehat{d}[v] \geq d(v)$, where $d(v)$ is the true shortest path distance to v, and a label is reduced only when an arc (u, v) is relaxed, that is, by setting $\widehat{d}[v] \leftarrow \widehat{d}[u] + w_{uv}$. A vertex is *complete* when $\widehat{d}[v] = d(v)$. Following [6], each algorithm maintains a *frontier*: a set of vertices such that the shortest path to every incomplete vertex passes through a complete frontier vertex. Repeatedly, a set Z of complete vertices is identified, the arcs leaving Z are relaxed, and the vertices of Z are replaced in the frontier by the out-neighbours just improved. The algorithms differ in how the set Z is identified and selected.

In Dijkstra's algorithm, Z is the single frontier vertex with the smallest label, which is necessarily complete. A priority queue over the whole frontier is therefore required, and when the frontier contains $\Theta(n)$ vertices each extraction incurs $\Theta(\log n)$ comparisons.

Both BMSSP algorithms instead solve the subproblem BMSSP(l, B, S): given a level l, a bound B and a frontier S, all vertices v with $d(v) < B$ whose shortest path visits S are to be completed. During a call, the part S_i of the frontier lying below a separator B_i is extracted, the subproblem $(l-1, B_i, S_i)$ is solved recursively, the arcs out of the returned complete set U_i are relaxed, and the process is repeated. A call may stop early (a *partial* execution) once enough vertices have been completed, returning a boundary $B' < B$ up to which its work is final. The whole problem is the initial call with $S = \{s\}$, $B = \infty$ at level $\lceil (\log n)/t \rceil$. Table I lists the components.

B. BMSSP-25: the algorithm of Duan, Mao, Mao, Shu and Yin

FindPivots of BMSSP-25 relaxes outward from S for k rounds, restricted to the labels below B, and collects every vertex it encounters in W. A key decision is then made from the sizes of W and S: if $|W| > k|S|$, the frontier is considered small relative to the remaining work, and every vertex of S is selected as a pivot. Otherwise, the tight arcs inside W form a forest, and only the roots in S of trees having at least k vertices are selected as pivots: a vertex whose shortest path leaves S through a smaller tree would already have been completed during the k rounds. The selected pivots are inserted into a specialized structure D that returns, for each Pull, at most $M = 2^{(l-1)t}$ keys of smallest value together with a separator. The vertices improved by relaxing out of U_i are then processed according to their new values: a vertex is inserted individually if its new value is at least B_i, and prepended as part of a batch if the value lies in $[B'_i, B_i)$. According to the analysis [5], FindPivots incurs a cost of $O(k)$ per vertex and level and the structure $O(t)$ per insertion, and the two are balanced by choosing $k = \lfloor \log^{1/3} n \rfloor$ and $t = \lfloor \log^{2/3} n \rfloor$.

C. BMSSP-26: the algorithm of Duan, Mao, Shu and Yin

The central observation underlying BMSSP-26 is that the k relaxation rounds make it possible to determine, for each vertex, the pivot tree to which it belongs, which is more information than the recursion strictly requires [6]. FindPivots now runs a Dijkstra search from each $x \in S$, terminating either once k vertices have been discovered or once a tree built by an earlier search is encountered, in which case the two trees are merged. If all reachable vertices are processed before k of them are found, x is added to a set Q and its vertices to W; these are excluded from the recursion and processed collectively at the end of the call. The remaining trees are partitioned by their edges into subtrees of between k and $3k$ vertices [19], the frontier vertices of each subtree form a group P_j, and only the vertex p_j of smallest key in each group is inserted into D. When p_j is pulled, the members of P_j below the separator are identified through a scan of the group and added to S_i. The structure is organized around a single search tree of blocks, each containing $M = t\,2^{(l-1)t}$ keys, and the remaining frontier of a call is returned in this form, allowing the caller to merge it in linear time. For a graph of maximum degree δ, a running time of $O(m(t + \log n \log t/(\delta t)))$ is obtained, which is minimized by $t = \lceil \sqrt{\log n \log\log n / \delta} \rceil$.

D. Preconditions

Distinct path lengths. Both analyses rely on the assumption that no two paths have the same length. To ensure this, both papers compare the labels lexicographically as tuples ⟨length, number of arcs, vertex, predecessor⟩. Each label therefore consists of four distinct fields, and more than one field may need to be examined during a comparison, in sharp contrast with Dijkstra's algorithm, in which comparisons are made on single numerical values.

Bounded degree. The analysis of BMSSP-25 is based on a graph in which every vertex has at most two incoming and two outgoing arcs, obtained by replacing each original vertex with a cycle of zero-weight arcs having one cycle vertex for each incident arc [5], [19]. The resulting graph contains at most $2m$ vertices and $3m$ arcs. BMSSP-26 generalizes the construction to cycles of $\lceil \Delta_v/(\delta - 2) \rceil$ vertices for a vertex having Δ_v incident arcs and requires $3 \leq \delta \leq m/n$. The value which is proposed for δ is $\min\{m/n, \log\log n\}/4$ [6], which reaches the required minimum of three only when $\log\log n \geq 12$, that is, for $n \geq 2^{4096}$; the authors note that the constants can be adjusted by the implementers. For every feasible input the prescription therefore comes down to $\delta = 3$ (or to no reduction at all when $m/n < 3$), and we use $\delta = 3$ in the formula for t everywhere.

E. Parameters at feasible sizes

Table II evaluates the prescribed parameters. For all of the sizes which fit in memory today BMSSP-25 uses $k \in \{2, 3\}$, so its FindPivots performs two or three relaxation rounds, and three or four recursion levels; BMSSP-26 also uses $k \in \{2, 3\}$, so each local search stops after two or three vertices, and its base case is a Dijkstra search over a few hundred vertices. The last two columns give the factor by which each of the bounds improves on $n \log n$ for sparse graphs when all of the hidden constants are set to one. Read this way the bound of BMSSP-25 is the smaller of the two until $(\log\log n)^3 < \log n$, that is,

TABLE I
COMPONENTS OF THE THREE ALGORITHMS

	Dijkstra [1], [2]	BMSSP-25 [5]	BMSSP-26 [6]
Running time	$O(m + n \log n)$	$O(m \log^{2/3} n)$	$O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$
Output	distances in sorted order	distances	distances
Complete set Z	minimum of the frontier	set U_i returned by a recursive call	set U_i returned by a recursive call
Frontier reduction	none	pivots: roots of shortest path trees with at least k vertices; at most $\lvert U\rvert/k$ of them	one representative (the minimum) per subtree of $\Theta(k)$ vertices of a forest spanning S
Pivot finding	-	k rounds of Bellman Ford relaxation from S: $O(k)$ per vertex	local Dijkstra searches that stop after k vertices, then a linear-time partition: $O(\log k)$ per vertex
Partial sorting	heap (total order)	two block sequences; INSERT in $O(\log(N/M))$, BATCHPREPEND, PULL	one search tree of blocks; INSERT in $O(\log(N/M))$, linear-time MERGE and PULL
Hand-back after a partial call	-	caller reinserts what it can reconstruct (BATCHPREPEND)	callee returns its remaining frontier D; caller merges it
Call stops when	frontier empty	$\lvert U\rvert \geq k2^{lt}$ or D empty	$\lvert U\rvert > t^3 2^{lt}$ or D empty
Base case ($l = 0$)	-	Dijkstra from one vertex, $k + 1$ vertices	Dijkstra from a set, $t^3 + 1$ vertices
Parameters	-	$k = \lfloor \log^{1/3} n \rfloor$, $t = \lfloor \log^{2/3} n \rfloor$	$t = \lceil \sqrt{\log n \log\log n/\delta} \rceil$, $k = \lceil t/\log t \rceil$
Degree assumption	none	in- and out-degree at most 2	degree at most δ, $3 \leq \delta \leq m/n$

TABLE II
PRESCRIBED PARAMETERS FOR CONCRETE n ($\delta = 3$). LEVELS = $\lceil \log_2 n/t \rceil$; GAIN = FACTOR BY WHICH THE BOUND OF THE ALGORITHM IMPROVES ON $n \log n$ WITH UNIT CONSTANTS

	BMSSP-25				BMSSP-26				
n	k	t	levels	gain	k	t	levels	t^3	gain
2^{10}	2	4	3	2.15	2	4	3	64	1.74
2^{20}	2	7	3	2.71	3	6	4	216	2.15
2^{30}	3	9	4	3.11	3	8	4	512	2.47
2^{40}	3	11	4	3.42	3	9	5	729	2.74
2^{64}	4	16	4	4.00	4	12	6	1 728	3.27

until $\log n$ exceeds 982. At any feasible size it is the constants that decide the comparison, and this is what the experiments measure.

IV. IMPLEMENTATION

All of the algorithms are implemented in C++17 in a single code base and share the graph representation (compressed sparse rows), the label type and the timing harness. The BMSSP-25 and BMSSP-26 implementations were written from the pseudocode of [5] and [6]; no code from other implementations was used.

Dijkstra. Two variants are considered: a binary heap [20] of (key, vertex) pairs with lazy deletion referred to as Dijkstra, and an indexed binary heap supporting DECREASEKEY, referred to as Dijkstra-DK. Both algorithms use standard path lengths. The former serves as the baseline for all comparisons and was found to be slightly faster than `std::priority_queue` on the two tested instances (SF and a random graph with 2^{20} vertices).

Labels. The BMSSP implementations store a 16-byte label (length, number of arcs, predecessor) for each vertex and compare the tuples which are described in Section III. Arc weights are 64-bit integers, so the sums are exact and the results of the different algorithms can be compared for equality.

Partial-sorting structures. Blocks are arrays having constant-time deletion by swapping with the last element. The block index is a red-black tree [21] (`std::map`) keyed by the upper bounds of the blocks, as proposed in both of the papers. A direct-address table for each recursion level maps a vertex to its block and position, as in [7]; this is possible because the calls on one level run one after the other. Selection of the element of a given rank, which is needed by PULL, block splits and BATCHPREPEND, uses quickselect which switches to median-of-medians pivots [22] once a linear work budget is exceeded [23], so that the worst case stays linear. The base case of BMSSP-26 uses a balanced search tree (`std::set`), as is prescribed by its pseudocode, and the local searches of both algorithms use binary heaps in place of Fibonacci heaps; with at most $k + 1$, respectively $k + \delta$, entries this does not change any bound which matters at the sizes that we consider.

Sets. The sets U which are returned by recursive calls are stored as contiguous ranges of one global array, so returning a set costs nothing and the set of a caller is the concatenation of the ranges of its children. Membership tests use time-stamped arrays.

Deviations from the pseudocode. Three places are implemented in a different way from the printed pseudocode. They can be switched on and off, and Section VI-A reports what each one of them changes. (i) In both algorithms a vertex is deleted from the D of the caller when a recursive call reports it as complete. For BMSSP-25 this is the repair of [7]. (ii) The base case of BMSSP-26 returns the smallest value which remains in its structure after the loop in place of the separator of its last PULL. (iii) Line 28 of the main procedure of BMSSP-26 selects a new representative only for the groups P_j which are still nonempty, as the accompanying text of [6] says; a group can be emptied by later vertices of the same U_i after it was scheduled for reselection.

Counters. A second build of the same sources increments counters for label or key comparisons, additions $\widehat{d}[u] + w_{uv}$, edge scans (split by the place where they occur), and structure operations. Comparisons include the ones made inside heaps, search trees and selection. The timing build has no instrumentation in it.

The implementations, the experiment drivers, the raw mea-

TABLE III

ROAD NETWORKS (ARCS COUNTED IN BOTH DIRECTIONS). FRONT.: LARGEST NUMBER OF VERTICES HELD AT ONCE BY DIJKSTRA'S HEAP, IN PERCENT OF n. LAST COLUMNS: PRESCRIBED $k/t/$LEVELS

	n	m	max deg.	m/n	front. (%)	BMSSP-25 $k/t/$levels	BMSSP-26
OL	6 105	14 070	5	2.30	1.7	2/5/3	2/4/4
TG	18 263	47 748	8	2.61	1.0	2/5/3	3/5/3
CAL	21 048	43 386	8	2.06	0.3	2/5/3	3/5/3
SF	174 956	446 002	8	2.55	0.3	2/6/3	3/5/4
NA	175 813	358 358	14	2.04	0.1	2/6/3	3/5/4

surements and the scripts which regenerate every table and figure of this paper from them are publicly available [24].

V. EXPERIMENTAL SETUP

Road networks. The real inputs are the five road networks of the SSTD 2005 collection [25], which were obtained from the public mirror [26]: Oldenburg (OL), San Joaquin County (TG), California (CAL), San Francisco (SF) and North America (NA); OL, TG and SF come from the generator data of Brinkhoff [27]. Each file lists a road segment once along with its Euclidean length. All of the five networks are connected when the segments are read as undirected, but a search which follows the segments only in their listed direction reaches as few as six vertices from vertex 0 of CAL, so every segment is entered as two opposite arcs. The lengths are given with six decimals and are multiplied by 10^6 to get exact integers. Table III lists the resulting graphs.

SNAP graphs. To cover larger real graphs of other kinds we add six graphs of the SNAP collection [28], all of which are also used in [17]: the web graphs web-Stanford and web-Google, the road networks of Pennsylvania, Texas and California, and the Pokec social network (Table IX). The road files list both of the directions of every road; all of the files are used as they are given. They have no weights, so every graph is run twice, once with unit weights as in [17] and once with uniform random integer weights in $[1, 2^{20}]$. Since a random vertex of a directed web graph may reach almost nothing, the sources are random vertices which reach at least a quarter of the graph.

Synthetic families. Four families having n from 2^{10} to 2^{22} extend the range of sizes and structures: (i) random digraphs having a random Hamiltonian cycle and $2n$ further uniformly random arcs ($m = 3n$); (ii) square grids having arcs in both directions; (iii) preferential-attachment graphs [29] with two links for each new vertex, entered in both directions, which have a heavy-tailed degree distribution; and (iv) hub graphs, which are random digraphs in which the source also has an arc to every other vertex whose weight is larger than every ordinary weight, so that the search starts with all of the n vertices in its frontier. Weights are uniform integers in $[1, 2^{20}]$ unless they are varied. Two sweeps on random digraphs having $n = 2^{18}$ vary the average out-degree from 2 to 32 and the largest weight from 1 (all of the paths of equal hop count tie) to 2^{30}.

TABLE IV

RANDOMIZED VALIDATION WITH THE PRESCRIBED PARAMETERS: NUMBER OF RUNS IN WHICH AN EVENT OCCURRED AT LEAST ONCE

Graphs	Variant	Runs	Wrong dist.	Stale key	Dupl. compl.	Bound viol.
$n \le 16$	BMSSP-25, as printed	200 000	0	5 293	4 788	0
	BMSSP-25, with deletion	200 000	0	5 293	0	0
	BMSSP-26, as printed	200 000	0	3 475	3 475	8 939
	BMSSP-26, both repairs	200 000	0	3 475	0	0
$n \le 400$	BMSSP-25, as printed	30 000	0	10 402	9 343	0
	BMSSP-25, with deletion	30 000	0	10 402	0	0
	BMSSP-26, as printed	30 000	0	10 510	10 510	1 435
	BMSSP-26, both repairs	30 000	0	10 510	0	0

Measures. We report the wall clock time of one SSSP computation and, from the instrumented build, the number of comparisons, additions and edge scans divided by m. A run includes the initialization of the labels but not the loading of the graph or the allocation of working memory, which is done once for each graph.

Protocol. On each of the road networks every algorithm is run from the same 20 uniformly random sources, nine times each; the SNAP instances use ten sources and three repetitions, and the synthetic instances three random sources (the hub for family (iv)) and five repetitions. Every run is a complete single source computation over the whole graph. The degree-reduction and parameter experiments use ten and five sources for each network. For each of the sources we take the median over the repetitions. Ratios to Dijkstra are formed for each source and summarized by their geometric mean. The distances of every timed run are compared with the ones of Dijkstra's algorithm. Unless stated otherwise the BMSSP algorithms run on the input graph as it is given, without degree reduction, with the parameters of Table III and with deviations (i) to (iii).

Platform. Apple M1 (8 cores, 8 GB), macOS 26.1, Apple clang 17.0.0 with `-O3`; all of the runs are single threaded.

VI. RESULTS

A. Validation and observations on the pseudocode

Every variant was run on 200 000 random graphs having at most 16 vertices and 30 000 random graphs having at most 400 vertices, drawn from six shapes (sparse, dense, paths with chords, stars, trees with extra arcs, grids) and five weight ranges including all-zero weights, and was compared with a Bellman Ford computation [30]. Each graph was solved with the prescribed parameters, with random small k and t which force partial executions, and after degree reduction with $\delta \in \{3, 4, 5\}$. None of the variants, repaired or as printed, returned a wrong distance in any of the 3 400 000 and 510 000 runs. Table IV summarizes what was recorded by the instrumentation with the prescribed parameters.

Stale keys. A key which is stored in D keeps the value it had when it was inserted, while the label of its vertex can decrease during a recursive call. For BMSSP-25 this is the defect reported in [7]: in our runs without deletion a vertex was reported as complete a second time within one call in 4 788

of the small and 9 343 of the medium instances, the smallest one having 9 vertices. The same thing happens in BMSSP-26. The proof of Lemma 9 in [6] (the lemma numbers refer to the proceedings version) states that the returned set U_i is not in D and does not need to be removed; in 3 475 small and 10 510 medium instances at least one vertex of U_i was still stored in the D of the caller when the subcall returned. Without deletion such a vertex is pulled again later with its old key and its already completed subtree is explored once again, so the sets U_i of one call are not disjoint (3 475 and 10 510 instances, the smallest having 12 vertices), which is contrary to Lemma 10 of [6], on which the running time analysis depends. Deleting the vertices of U_i from D after the merge removes every one of these events. We have not contacted the authors of [6]; the statement here is limited to what is shown by the instrumented runs.

Boundary returned by the base case of BMSSP-26. Algorithm 4 of [6] returns the separator which is delivered by its last Pull, and this is computed before the arcs of the pulled vertex are relaxed. If the loop then ends because t^3+1 vertices have been extracted, these relaxations may have inserted values below the returned B', which contradicts the requirement $B' \le d_B[D]$ of Lemma 9, or the structure may be nonempty even though $B' = B$ signals a full execution. A directed path having t^3+2 vertices which is searched from its first vertex is the simplest example: the last Pull empties the structure and returns B, and relaxing the pulled vertex then inserts its successor. The event occurred in 8 939 small and 1 435 medium instances (smallest: 10 vertices). Returning the smallest remaining value after the loop restores the invariant.

On the road networks these events are rare. Without deletion, repeated completions come to at most 1.0% of n for BMSSP-25 and at most 0.9% for BMSSP-26, and the running times of the printed and the repaired variants agree within measurement noise. The repairs are important for the validity of the analysis and not for the speed on these inputs.

B. Road networks: running time

Table V gives the main comparison. Dijkstra's algorithm is faster than both of the BMSSP algorithms on all of the five networks from every source: by a factor between 3.5 and 8.0 against BMSSP-25 and between 4.7 and 6.7 against BMSSP-26. The two Dijkstra variants are within about ten percent of each other. BMSSP-26 is the faster one of the two new algorithms on one network only (CAL) and almost ties on NA; these are the two networks having the smallest average degree (Table III), and on them BMSSP-25 requires the most comparisons (Table VI). Over the individual start vertices the ratio ranges from 3.2 to 9.5 for BMSSP-25 and from 4.3 to 7.3 for BMSSP-26.

The largest ratios are a property of two particular inputs and not of road networks in general. In CAL and NA 93.5% and 94.8% of the vertices have exactly two neighbours, so these networks are long chains of shape points which are joined at a small number of junctions, against 17% to 53% in the other three. On the two chain-like networks BMSSP-25 is 6.7 to 8.0 times slower than Dijkstra, on the other three 3.5 to 5.1 times, and on the three much larger SNAP road networks (Section VI-H) 2.6 to 3.0 times. The latter values are close to the factor of three to four which is reported by [7] for large road networks. Size also matters: on instances having a few thousand vertices all of the ratios are higher, in our measurements (Fig. 3) as well as in theirs.

TABLE V
Road networks: median time of one SSSP computation in milliseconds, and ratio to Dijkstra (geometric mean of the per-source ratios). Dijkstra: binary heap with lazy deletion; Dijkstra-DK: indexed heap with DecreaseKey

	Dijkstra	Dijkstra-DK		BMSSP-25		BMSSP-26	
	ms	ms	ratio	ms	ratio	ms	ratio
OL	0.220	0.223	1.02	1.083	5.08	1.285	5.86
TG	0.847	0.846	1.00	3.492	4.08	3.969	4.68
CAL	0.592	0.620	1.05	4.775	7.95	3.153	5.31
SF	8.495	9.157	1.07	30.002	3.50	40.222	4.68
NA	5.609	6.295	1.11	36.963	6.68	39.088	6.73

TABLE VI
Road networks: comparisons (cmp.), additions (add.) and edge scans (scans) per arc, mean over sources. Dijkstra's algorithm performs exactly one addition and one edge scan per arc

	Dijkstra	BMSSP-25			BMSSP-26		
	cmp.	cmp.	add.	scans	cmp.	add.	scans
OL	6.66	13.05	3.94	4.71	14.30	5.13	5.13
TG	6.97	11.58	3.67	4.56	12.07	4.03	4.03
CAL	6.12	16.29	4.43	4.77	12.76	4.02	4.02
SF	7.99	11.12	3.32	4.25	13.06	5.08	5.08
NA	7.41	15.53	4.13	4.68	15.64	5.06	5.06

C. Road networks: operation counts

Table VI distinguishes the total algorithmic work from the cost of each individual operation. Dijkstra's algorithm scans every arc exactly once and performs one addition per arc. In comparison, BMSSP-25 scans each arc between 4.3 and 4.8 times, while BMSSP-26 scans each arc between 4.0 and 5.1 times. Since every scan involves evaluating $\widehat{d}[u] + w_{uv}$, the number of additions increases by 3.3 to 4.4 times for BMSSP-25 and by 4.0 to 5.1 times for BMSSP-26. More importantly, both algorithms also perform *more comparisons* than Dijkstra's algorithm on these networks. BMSSP-25 requires 1.39 to 2.66 times as many comparisons, whereas BMSSP-26 requires 1.63 to 2.15 times as many. As shown in Table III, Dijkstra's heap never contains more than 1.7% of the vertices in a road network. Consequently, there is relatively little ordering work to eliminate, and the savings achieved are outweighed by the additional comparisons resulting from repeated relaxations. Section VI-G demonstrates that this situation changes when the algorithms are applied to graphs with a wide frontier.

Fig. 1 illustrates where the arc scans take place and highlights the changes introduced by BMSSP-26. In BMSSP-25, FindPivots contributes more than one scan per arc across every network. The relaxation rounds within this procedure touch each arc approximately as many times as Dijkstra's algorithm does during its entire execution. In BMSSP-26, however, the local searches account for only a small fraction

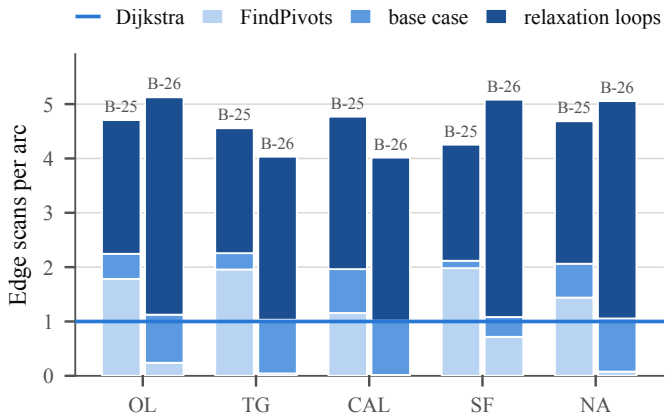


Fig. 1. Edge scans per arc on the road networks for BMSSP-25 (bars marked B-25) and BMSSP-26 (B-26), split by where they occur. Dijkstra's algorithm scans every arc exactly once.

of this cost, indicating that the refinement achieves its intended purpose. These savings are nevertheless offset by the additional work in the relaxation loops. At every level of recursion, the algorithm relaxes all arcs leaving the sets it receives. With the prescribed parameters, BMSSP-26 has one more recursion level than BMSSP-25 on OL, SF and NA (Table III). The additional topmost level simply passes the single source downward and then scans every arc once more on the way back up.

When the time ratio is divided by the ratio of the combined number of comparisons and additions, the resulting value represents the relative cost per counted operation. This cost ranges from 2.1 to 2.9 for BMSSP-25 and from 2.3 to 2.7 for BMSSP-26. The overall slowdown can therefore be attributed to two factors of comparable magnitude. First, under the comparison-addition model, BMSSP-25 performs 1.6 to 2.9 times more work, while BMSSP-26 performs 2.0 to 2.5 times more work. Second, each operation incurs additional costs that are not captured by the model. These include the use of four-field labels instead of single numbers, the bookkeeping required to manage blocks and tables, and less regular memory access patterns.

D. Degree reduction

Table VII presents the impact of applying the degree-reduction transformation to the graph, as assumed in the theoretical analyses. With $\delta = 3$, the transformed graph contains $2m$ vertices and $3m$ arcs, resulting in a vertex count that is 4.1 to 5.2 times that of the original graph. On this transformed input, BMSSP-25 becomes 2.5 to 5.6 times slower, while BMSSP-26 becomes 2.0 to 3.2 times slower than when operating on the original graph. Dijkstra's algorithm also experiences a slowdown of several times on the transformed input. Since the road networks already have a maximum degree of at most 14, the transformation provides no benefit for these networks. This observation is consistent with the findings for BMSSP-25 reported in [7]. Increasing the bound to $\delta = 4$ reduces the performance penalty for BMSSP-26, but does not eliminate it.

TABLE VII
DEGREE REDUCTION: GROWTH OF THE GRAPH FOR $\delta = 3$ AND TIME RELATIVE TO DIJKSTRA ON THE ORIGINAL GRAPH

	growth		Dijkstra	BMSSP-25		BMSSP-26		
	n	m	δ=3	orig.	δ=3	orig.	δ=3	δ=4
OL	4.61	3.00	3.8	5.0	21.9	5.9	11.9	9.8
TG	5.23	3.00	3.9	4.1	21.1	4.6	14.0	9.9
CAL	4.12	3.00	3.6	7.8	19.5	5.3	13.7	8.7
SF	5.10	3.00	4.6	3.6	20.4	4.7	15.0	10.9
NA	4.08	3.00	4.5	6.6	24.9	6.7	19.5	11.7

E. Sensitivity to the parameters k and t

The parameters k and t are chosen based on asymptotic considerations, but both papers leave the specific constant values open. Fig. 2 presents the running times on SF for all combinations of $k \in \{1, \ldots, 6\}$ and $t \in \{2, \ldots, 10\}$, with the number of recursion levels determined by t. Three observations hold for both SF and NA. First, setting $k = 1$ consistently produces poor performance. Without frontier reduction, every vertex in the frontier proceeds through the recursion, making the algorithms significantly slower. On SF, they run up to 14.1 times slower than Dijkstra's algorithm for BMSSP-25 and up to 12.8 times slower for BMSSP-26. Second, the prescribed parameter values are effective, but they do not deliver the best performance. For BMSSP-25, increasing k generally improves performance, which aligns with the structure of these networks. Their average degree ranges from 2.0 to 2.6, indicating that they consist largely of chains. With a larger k, additional relaxation rounds can process longer chains within FindPivots. On SF, the fastest configuration is $k = 5$, $t = 6$, which runs 12% faster than the prescribed setting. On NA, the best configuration is $k = 6$, $t = 5$, achieving a 39% speedup. For BMSSP-26, the fastest configurations use $t = 9$ on both networks, reducing the number of levels from four to two. These settings improve performance by 11% on SF and 31% on NA. Third, changing the parameter settings does not alter the overall conclusion. Across the entire parameter grid, the best running time ratios relative to Dijkstra's algorithm are 3.05 and 4.09 on SF, and 3.94 and 4.72 on NA, for BMSSP-25 and BMSSP-26, respectively.

The topmost recursion level in BMSSP-26 warrants further explanation. The call immediately below the top level can process up to $t^3 2^{(L-1)t}$ vertices, where $L = \lceil \log n/t \rceil$. This capacity is at least n whenever $t^3 \geq 2^t$, which holds for $t \leq 9$. Consequently, for every n up to 2^{40} (Table II), the top-level call extracts the source, receives the entire vertex set V from its single recursive subcall, and then relaxes every arc once more without producing any further changes. This behavior is reflected in the operation counts: BMSSP-26 scans every arc once in a base case and once at each recursion level.

F. Scaling on synthetic families

Fig. 3 compares the running times of the algorithms relative to Dijkstra's algorithm as the graph size n increases from



Fig. 2. Running time relative to Dijkstra on SF for every combination of k and t (lighter is faster). Solid box: parameters prescribed by the respective paper; dotted box: fastest combination.

2^{10} to 2^{22} for random and grid graphs, and up to 2^{21} for preferential-attachment and hub graphs. On random directed graphs, the relative running time decreases steadily as the graph size increases. For BMSSP-25, the ratio drops from 5.9 at $n = 2^{10}$ to 1.5 at $n = 2^{22}$, while BMSSP-26 improves from 6.8 to 1.8 over the same range. Preferential-attachment graphs show a similar pattern, with ratios of 1.2 for BMSSP-25 and 2.0 for BMSSP-26 at $n = 2^{21}$. The heavy-tailed degree distribution of these graphs does not produce any noticeable difference in performance. Grid graphs show a different trend, as the improvement levels off at an earlier stage. Starting at $n = 2^{14}$, both algorithms remain between 2.7 and 3.8 times slower than Dijkstra's algorithm. On hub graphs, BMSSP-25 improves to a ratio of 1.7, whereas BMSSP-26 remains 3.9 times slower than Dijkstra at the largest tested graph size. Across all 44 synthetic instances, BMSSP-26 is slower than BMSSP-25, and neither of the two newer algorithms outperforms Dijkstra's algorithm anywhere within the tested range. The performance curves are not smooth because the prescribed values of k, t and the number of recursion levels are rounded and change in discrete steps. For graphs containing more than 2^{20} vertices, the two Dijkstra variants also begin to diverge in performance. The DecreaseKey variant becomes up to 2.1 times slower, suggesting that, at these graph sizes, all algorithms are increasingly affected by memory-hierarchy effects that are not captured by the cost model.

Fig. 4 presents the same instances using the cost model described in the theorems, revealing different trends across graph families. On random and preferential-attachment graphs, the number of comparisons performed by Dijkstra's algorithm per arc increases linearly with $\log n$. With the lazy heap, this count rises by 0.80 and 0.67 comparisons per arc for each doubling of n, respectively. With the DecreaseKey variant, the corresponding increases are 0.67 and 0.50. In contrast, the comparison counts for the BMSSP algorithms remain nearly constant. On random graphs, BMSSP-25 performs between 10 and 12 comparisons per arc, while BMSSP-26 performs between 12 and 15. As a result, the curves eventually cross. For random graphs, BMSSP-25 requires fewer comparisons than binary-heap Dijkstra at every tested size from $n = 2^{14}$ onward. For preferential-attachment graphs, this advantage appears from $n = 2^{16}$ onward. When compared with the DecreaseKey variant, BMSSP-25 achieves the same advantage from $n = 2^{17}$ for random graphs and $n = 2^{20}$ for preferential-attachment graphs. Considering the combined number of comparisons and additions, BMSSP-25 falls below Dijkstra's count from $n = 2^{16}$ and $n = 2^{18}$, respectively. BMSSP-26 also eventually requires fewer comparisons than Dijkstra's algorithm, with its comparison count dropping below Dijkstra's at $n = 2^{20}$ in both graph families. These results show the asymptotic advantage of the new algorithms emerging at practical graph sizes within the cost model. Although the theoretical advantage is established by the cost model, it does not necessarily translate into faster execution. For example, at $n = 2^{20}$ on random graphs, BMSSP-25 performs 11.9 comparisons and additions per arc, compared with 16.5 for Dijkstra's algorithm, yet it still takes 1.7 times as long to run. On grid and hub graphs, the comparison curves do not cross, for two different reasons discussed below.

G. Graph structure: frontier width and maximum degree

Two properties of the input graphs help explain the performance differences across the graph families.

Frontier width. We measure the width of Dijkstra's frontier by the maximum number of vertices held in its indexed heap at any point during execution. On road networks, this value ranges from 0.1% to 1.7% of n (Table III). On grid graphs, the frontier width decreases from 7.5% at $n = 2^{10}$ to 0.1% at $n = 2^{22}$. Because the heap remains small, extracting vertices is inexpensive, leaving little ordering work for a more sophisticated algorithm to eliminate. Consequently, Dijkstra's algorithm performs fewer comparisons than both new algorithms at every tested graph size. In contrast, on

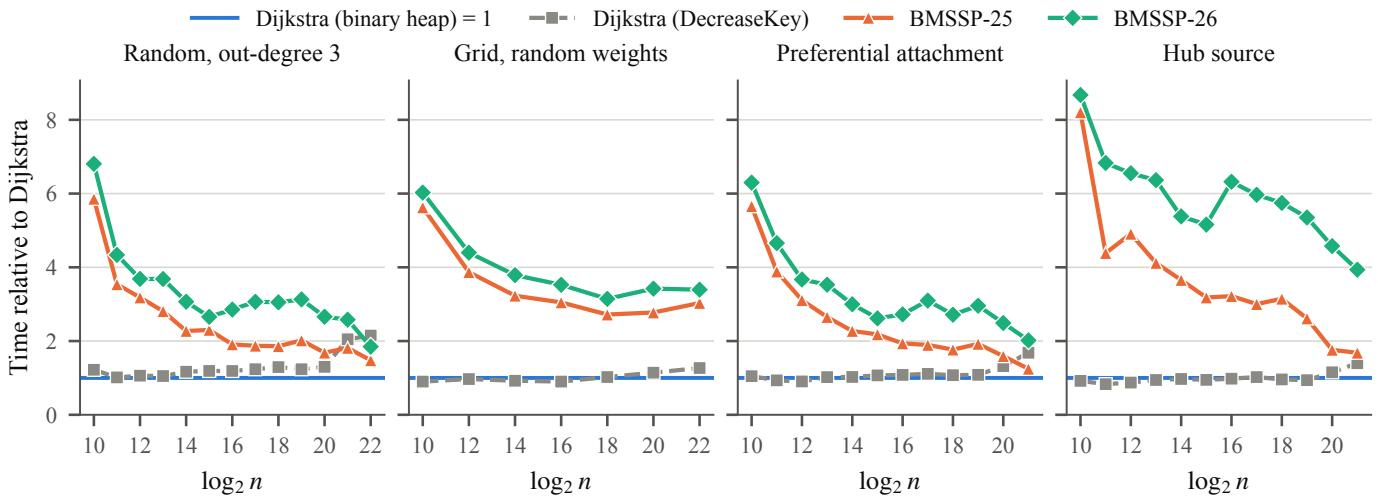


Fig. 3. Running time relative to Dijkstra with a binary heap on four synthetic families as a function of n (geometric mean over sources).

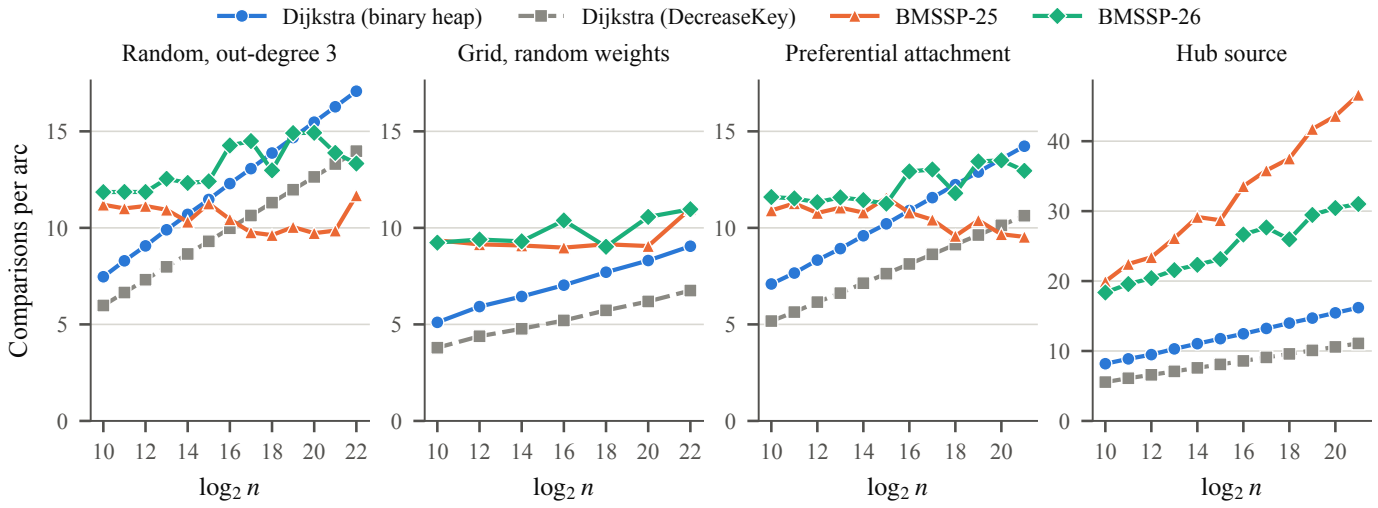


Fig. 4. Comparisons per arc on the synthetic families as a function of n. The hub family has its own vertical scale.

random and preferential-attachment graphs, the frontier contains approximately 33% and 51% of the vertices, respectively, across all tested sizes. Dijkstra's algorithm therefore incurs the full logarithmic factor associated with maintaining the heap, allowing the comparison counts of the new algorithms to fall below its count. The running time results follow a similar pattern: the lowest ratios relative to Dijkstra's algorithm occur on random and preferential-attachment graphs, while higher ratios are observed on grids and road networks.

Maximum degree. On hub graphs, the frontier reaches its maximum possible width because all n vertices receive labels as soon as the source vertex is settled. Despite this, both BMSSP algorithms perform more comparisons than Dijkstra's algorithm. As shown in Fig. 4 (right), BMSSP-25 requires 2.4 to 2.9 times as many comparisons, with the gap increasing as n grows, while BMSSP-26 requires 1.9 to 2.2 times as many. The operation counters help explain this behavior. After the first base case, all $n - 1$ arcs of the hub are relaxed together. In BMSSP-25, these arcs fall within the range that is batch-prepended at level one. Since each block contains a single key, the algorithm orders them through repeated median selection. In BMSSP-26, the keys are inserted into the base case's search tree and then passed upward through merge operations. In both cases, the algorithm must order the entire frontier, and the cost per key is higher than that of a heap. The theoretical analyses exclude this situation through their degree bound, which limits the number of keys that a single completed vertex can generate. Table VIII confirms that the missing degree bound is responsible for this behavior. After degree reduction with $\delta = 3$, the instance contains eight times as many vertices and satisfies the required precondition. On this transformed graph, BMSSP-26 performs fewer than half as many comparisons as Dijkstra's algorithm: 55 compared with 118 per arc of the original graph. BMSSP-25 also performs fewer comparisons,

TABLE VIII

HUB GRAPH WITH $n = 2^{18}$ AS GIVEN AND AFTER DEGREE REDUCTION. CMP.: COMPARISONS PER ARC OF THE ORIGINAL GRAPH; TIME: RELATIVE TO DIJKSTRA ON THE GRAPH AS GIVEN

		Dijkstra		BMSSP-25		BMSSP-26	
Graph	vertices	cmp.	time	cmp.	time	cmp.	time
as given	262 144	14.0	1.0	37.5	2.9	26.0	6.4
$\delta = 3$	2 097 150	118.2	6.7	97.9	19.3	55.4	17.1

requiring 98 per arc of the original graph. However, this advantage comes at a substantial computational cost. Dijkstra's algorithm on the original graph requires only 14 comparisons per arc, while the degree-reduction transformation alone makes it 6.7 times slower. Even on the transformed graph, BMSSP-25 and BMSSP-26 take 2.9 and 2.5 times as long, respectively, as Dijkstra's algorithm running on that same transformed graph.

H. Web, road and social graphs

Table IX and Fig. 5 extend the evaluation to six larger real world graphs. Across all twelve instances, Dijkstra's algorithm delivers the fastest running time. BMSSP-25 runs between 1.3 and 3.2 times slower than Dijkstra's algorithm, whereas BMSSP-26 is between 1.9 and 4.5 times slower, and BMSSP-26 outperforms BMSSP-25 on none of the tested instances. Among these results, the closest performance to Dijkstra's algorithm is observed on Pokec with random weights, the densest graph having the widest frontier. In case of having unit weights, the setup in which [17] reports its simplified solver ahead of Dijkstra, the new algorithms are 2.2 to 3.2 (BMSSP-25) and 3.0 to 4.5 (BMSSP-26) times slower; we observed no practical advantage on any of the graphs that we chose.

The counts are similar to the pattern of Section VI-G. The frontiers of the three road networks are at most 0.3% of n, and both the published algorithms require more comparisons than Dijkstra's algorithm. The web and social graphs having frontiers between 13% and 50% of n; with random weights BMSSP-25 need fewer comparisons than the lazy-heap baseline on two of them (Google and Pokec), the two of the largest. Two qualifications apply. Measured against the DECREASEKEY variant, whose heap never exceeds n entries therefore is much cheaper on graphs which are dense, BMSSP-25 needs more comparisons on all of the twelve instances. And with unit weights, where Dijkstra's algorithm needs few comparisons per arc, the new algorithms need more of them on every graph (1.03 to 1.65 times for BMSSP-25).

I. Density and weight range

Two sweeps on random digraphs with $n = 2^{18}$ vary further properties. On increasing the average out-degree from 2 to 8, the ratio to Dijkstra from 2.2 to 1.5 for BMSSP-25 and from 3.6 to 2.3 for BMSSP-26 lowers; beyond that the ratio remains roughly constant. Graphs which are denser spend a larger share of the time in plain edge scans, which is performed by all algorithms, but the BMSSP algorithms scan each arc three to four times, because of which the ratio does not approach one. The weight range has little influence: from unit weights, where all paths with the same number of arcs tie and the tie-breaking rule decides many comparisons, to weights up to 2^{30} the ratio stays between 1.9 and 2.3 for BMSSP-25 and between 2.8 and 3.8 for BMSSP-26.

TABLE IX

SNAP GRAPHS. FRONT.: PEAK FRONTIER OF DIJKSTRA IN PERCENT OF n (RANDOM WEIGHTS). FOR EACH OF THE TWO NEW ALGORITHMS: RUNNING TIME RELATIVE TO DIJKSTRA WITH UNIT WEIGHTS (UNIT) AND WITH RANDOM WEIGHTS (RAND.), AND COMPARISONS RELATIVE TO DIJKSTRA WITH RANDOM WEIGHTS (CMP.)

	n	m	front.	BMSSP-25			BMSSP-26		
	(10^3)	(10^6)	(%)	unit	rand.	cmp.	unit	rand.	cmp.
Stanford	282	2.31	13.1	3.2	3.1	1.20	4.5	4.3	1.52
Google	876	5.11	15.5	2.2	2.5	0.79	3.0	3.7	1.23
roadPA	1088	3.08	0.3	2.9	2.6	1.10	3.8	3.4	1.25
roadTX	1380	3.84	0.3	3.0	2.6	1.10	3.8	3.5	1.24
roadCA	1965	5.53	0.3	2.8	2.6	1.09	3.5	3.3	1.22
Pokec	1633	30.62	50.3	2.3	1.3	0.80	3.5	1.9	1.29

J. Memory

The BMSSP implementations keeps a one direct-address table for each recursion level in addition to the labels. On the random graph with $n = 2^{22}$ the peak resident set of the whole process, which includes the graph and the reference solution, is 501 MB for Dijkstra, 859 MB for BMSSP-25 and 1170 MB for BMSSP-26, a factor of 1.7 and 2.3.

VII. DISCUSSION

What BMSSP-26 gains over BMSSP-25? The redesigned FINDPIVOTS does what is promised. On the road networks it accounts for 0.02 to 0.72 edge scans per arc, against 1.16 to 1.98 for the relaxation rounds of BMSSP-25. The savings are lost by the side effects of the new parameter setting. The value of t is smaller than in BMSSP-25 (Table II), which gives more levels, and every level relaxes all arcs once: the relaxation loops cost 3.0 to 4.0 scans per arc against 2.1 to 2.8. In addition the base case is now a search-tree Dijkstra over up to $t^3 + 1$ vertices instead of a heap with a small number of entries. In total BMSSP-26 performs more additions than BMSSP-25 on four of the five road networks and more comparisons on four of them, and it was slower on four of the five road networks and on all synthetic instances. When t is chosen freely the fastest configurations of BMSSP-26 have two levels (Fig. 2), which suggests that the balance between t and the cost of FINDPIVOTS that is optimal asymptotically is not the practical at these sizes.

Where the time goes. For the narrow-frontier inputs the new algorithms lose twice: they execute 1.6 to 2.9 (BMSSP-25) and 2.0 to 2.5 (BMSSP-26) times as many comparisons and additions as Dijkstra's algorithm on the road networks, and each one of these counted operations costs them 2.1 to 2.9 and 2.3 to 2.7 times as much time. For wide-frontier inputs of bounded degree the first handicap disappears as n grows, but the second remains and keeps the new algorithms behind. It is tied to features of the design that the cost model does not penalize: every arc is scanned three to five times instead of just once, labels are four-field tuples instead of just

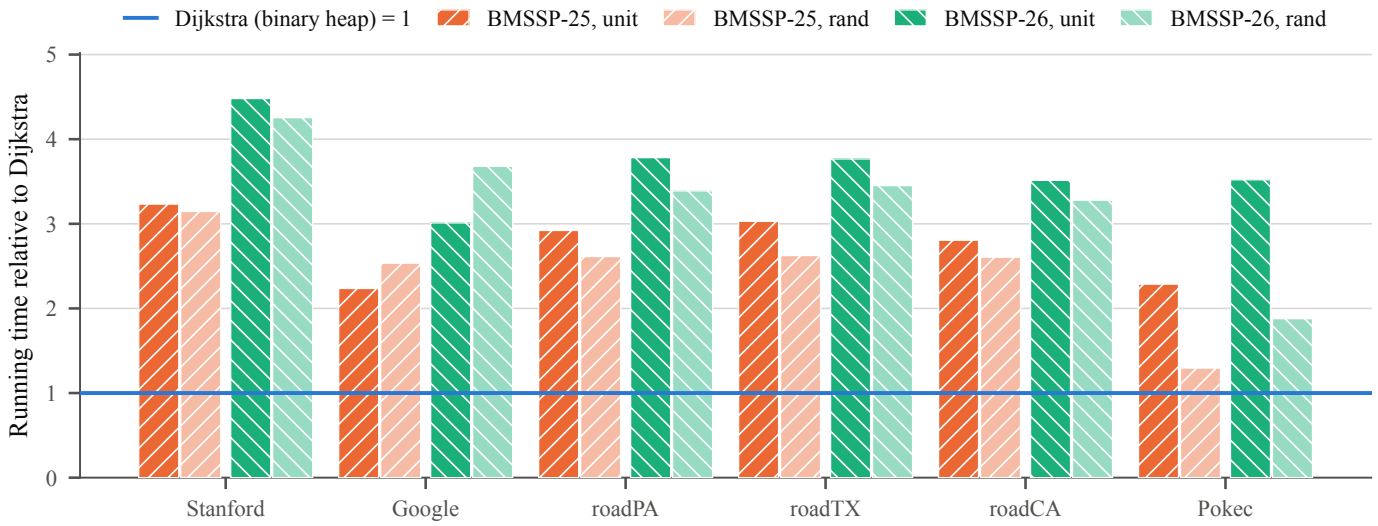


Fig. 5. Running time relative to Dijkstra with a binary heap on the six SNAP graphs, with unit and with random weights (geometric mean over ten sources). Dijkstra's algorithm is the fastest on all twelve instances.

single numbers, and the block structures add indirection and bookkeeping. Reducing the number of passes over the arcs, for instance by skipping the redundant topmost level of BMSSP-26, attacks the right term, but our parameter sweep indicates gains of tens of percent and not the factors.

Relation to earlier measurements. Our conclusion agrees with every study of an implementation that follows the published algorithm: [7] report Dijkstra to be three to four times faster on large instances, [8] find it faster on Lightning Network graphs, and the solver of [17] that follows the paper is reported as two to four times slower. Since our BMSSP-25 has the same structure as the worst case implementation of [7], which uses the same parameters, per-level tables and deletion repair, its published results also provide a useful check that our implementation is not unusually slow. For random graphs with $m = 3n$, the authors report running times of 10.7 ms for Dijkstra's algorithm and 38.2 ms for BMSSP at $n = 2^{16}$, 421 ms and 1286 ms at $n = 2^{20}$, and 2023 ms and 7703 ms at $n = 2^{22}$, respectively, using an Intel i5-10400F. On an Apple M1, we measure 6 ms and 11 ms, 271 ms and 454 ms, and 1866 ms and 2805 ms for the same graph sizes. Both of our implementations run faster in absolute terms, with the BMSSP implementation showing the larger improvement. As a result, our running time ratios of 1.9, 1.7 and 1.5 are lower than the corresponding ratios reported in [7], which are 3.6, 3.1 and 3.8.

The two sets of results also differ in how the ratios change with graph size. In [7], the ratio decreases from 25 at $n = 2^7$ to approximately three at $n = 2^{14}$, after which it remains between 2.8 and 3.8 up to $n = 2^{25}$. In our experiments, the ratio continues to decline gradually up to $n = 2^{22}$. However, the experiments use different machines and graph generators, and we did not run the implementation from [7]. Therefore, we cannot determine whether these differences arise from the implementations or the hardware, and the comparison should be treated as indicative rather than conclusive. The practical advantages over Dijkstra's algorithm reported in the studies we are aware of, [17] and [18], concern simplified variants operating on unit-weight graphs (Section II). In contrast, Section VI-H shows that the published algorithms do not achieve such an advantage on the same graphs.

Threats to validity. All runtime measurements were collected on a single machine using a single compiler. The implementations follow the pseudocode and incorporate sensible engineering choices, including flat arrays, direct-address tables, and avoiding per-call memory allocation. However, they have not been optimized to their full potential. In addition, the base case of BMSSP-26 uses a node-based search tree because this is what its pseudocode specifies. The operation counts are only partly independent of these implementation choices: they also depend on the heap and selection routines being used. For example, a heap that requires fewer comparisons per operation would shift Dijkstra's curves downward. All edge weights are integers, and the effects of rounding when using floating point weights have not been investigated. The largest instances contain 2^{22} vertices, which is still far from the scale at which the asymptotic bounds begin to separate. Therefore, we make no claims about the instance size at which a crossover in running time might occur. Finally, the two observations discussed in Section VI-A are based on our interpretation of the pseudocode in [6] and have not been verified by the authors of that work.

VIII. Conclusion

We compared Dijkstra's algorithm with the two algorithms proposed by Duan et al. that break the sorting barrier for directed graphs. The evaluation used independent implementations, eleven real graphs, four synthetic graph families, and operation counts under the comparison-addition model. Across every instance, Dijkstra's algorithm with a binary heap was the

fastest of the three algorithms, with speed advantages ranging from factors of 1.2 to 8.7. The results under the cost model reveal a more nuanced picture. When Dijkstra's frontier is narrow, as it is on road networks and grids, the new algorithms perform more comparisons and additions at every tested size. However, when the frontier is wide and vertex degrees are bounded, BMSSP-25 performs fewer comparisons than lazy-heap Dijkstra once the graph reaches a few tens of thousands of vertices. This advantage appears at sizes ranging from 10^5 to 10^6 vertices when compared with the DecreaseKey variant. BMSSP-26 achieves a similar advantage at approximately one million vertices. These results show that the asymptotic advantage is real and observable on synthetic graphs. On real web and social graphs, however, this advantage appears only against the lazy-heap implementation and only on the two largest instances. It has not yet translated into a runtime advantage, because every arc is scanned multiple times, and each operation incurs a higher cost. BMSSP-26 achieves the lower pivot-finding cost predicted by its analysis, but with the prescribed parameter settings, this improvement was not enough to make it faster than BMSSP-25 on most inputs. The validation also revealed two cases in which the published BMSSP-26 pseudocode deviates from its stated invariants. Both issues are straightforward to fix, and neither affected any computed distance. Several useful directions for future work emerge from these findings. First, parameter-selection rules could be developed using measured constants rather than relying solely on asymptotic balance. Second, BMSSP-26 could be implemented without the redundant top level and the node-based base case. Finally, further measurements should be conducted on machines capable of handling graphs large enough to extend the comparison-count analysis beyond 2^{22} vertices.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *J. ACM*, vol. 34, no. 3, pp. 596–615, Jul. 1987, doi: 10.1145/28869.28874.

[3] J. R. Driscoll, H. N. Gabow, R. Shrairman, and R. E. Tarjan, "Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation," *Commun. ACM*, vol. 31, no. 11, pp. 1343–1354, Nov. 1988, doi: 10.1145/50087.50096.

[4] B. Haeupler, R. Hladík, V. Rozhoň, R. E. Tarjan, and J. Tětek, "Universal optimality of Dijkstra via beyond-worst-case heaps," in *Proc. IEEE 65th Annu. Symp. Found. Comput. Sci. (FOCS)*, 2024, pp. 2099–2130, doi: 10.1109/FOCS61266.2024.00125.

[5] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, "Breaking the sorting barrier for directed single-source shortest paths," in *Proc. 57th Annu. ACM Symp. Theory Comput. (STOC)*, 2025, pp. 36–44, doi: 10.1145/3717823.3718179.

[6] R. Duan, X. Mao, X. Shu, and L. Yin, "A faster directed single-source shortest path algorithm," in *Proc. 53rd Int. Colloq. Automata, Lang., Program. (ICALP)*, LIPIcs, vol. 374, 2026, pp. 81:1–81:23, doi: 10.4230/LIPIcs.ICALP.2026.81.

[7] L. Castro, T. Clementino, and R. de Freitas, "Implementation and experimental analysis of the Duan et al. (2025) algorithm for single-source shortest paths," 2026, arXiv:2511.03007v3.

[8] D. Valko, R. Paranjpe, and J. Marx Gómez, "An empirical evaluation of BMSSP and Dijkstra's algorithm for the Lightning Network," 2026, arXiv:2509.13448v2.

[9] C. Makowski, W. Guter, T. Russell, and A. Saragih, "BMSSPy: A Python package and empirical comparison of bounded multi-source shortest path algorithm," MIT Center Transp. Logistics, Cambridge, MA, USA, Res. Paper 2025/034, Nov. 2025. [Online]. Available: https://ssrn.com/abstract=5777186

[10] B. V. Cherkassky, A. V. Goldberg, and T. Radzik, "Shortest paths algorithms: Theory and experimental evaluation," *Math. Program.*, vol. 73, no. 2, pp. 129–174, May 1996, doi: 10.1007/BF02592101.

[11] M. Thorup, "Undirected single-source shortest paths with positive integer weights in linear time," *J. ACM*, vol. 46, no. 3, pp. 362–394, May 1999, doi: 10.1145/316542.316548.

[12] S. Pettie and V. Ramachandran, "A shortest path algorithm for real-weighted undirected graphs," *SIAM J. Comput.*, vol. 34, no. 6, pp. 1398–1431, 2005, doi: 10.1137/S0097539702419650.

[13] R. Duan, J. Mao, X. Shu, and L. Yin, "A randomized algorithm for single-source shortest path on undirected real-weighted graphs," in *Proc. IEEE 64th Annu. Symp. Found. Comput. Sci. (FOCS)*, 2023, pp. 484–492, doi: 10.1109/FOCS57990.2023.00035.

[14] S. Yan, "Lossless derandomization for undirected single-source shortest paths and approximate distance oracles," in *Proc. SIAM Symp. Simplicity Algorithms (SOSA)*, 2026, pp. 404–412, doi: 10.1137/1.9781611978964.32.

[15] A. Kadria and L. Roditty, "A faster undirected single-source shortest path algorithm," 2026, arXiv:2609.15247.

[16] L. Castro, T. Clementino, and R. de Freitas. *bmssp: Header-only C++ implementation of the shortest-path BMSSP algorithm*. GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/lcs147/bmssp

[17] M. Ryu. *bmssp-python: Python implementation of the BMSSP shortest path algorithm*. (2025). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/bzantium/bmssp-python

[18] J. Webb. *DunMaoSSSP: A Rust implementation of the Duan–Mao et al. algorithm for single-source shortest paths*. (2025). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/alphastrata/DunMaoSSSP

[19] G. N. Frederickson, "Data structures for on-line updating of minimum spanning trees," in *Proc. 15th Annu. ACM Symp. Theory Comput. (STOC)*, 1983, pp. 252–257, doi: 10.1145/800061.808754.

[20] J. W. J. Williams, "Algorithm 232: Heapsort," *Commun. ACM*, vol. 7, no. 6, pp. 347–348, Jun. 1964.

[21] L. J. Guibas and R. Sedgewick, "A dichromatic framework for balanced trees," in *Proc. 19th Annu. Symp. Found. Comput. Sci. (SFCS)*, 1978, pp. 8–21, doi: 10.1109/SFCS.1978.3.

[22] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest, and R. E. Tarjan, "Time bounds for selection," *J. Comput. Syst. Sci.*, vol. 7, no. 4, pp. 448–461, Aug. 1973, doi: 10.1016/S0022-0000(73)80033-9.

[23] D. R. Musser, "Introspective sorting and selection algorithms," *Softw. Pract. Exper.*, vol. 27, no. 8, pp. 983–993, Aug. 1997.

[24] Anmol, Tanmay Arora, Adhyyan, and Pahul Veer Singh Gogna. *bmssp-vs-dijkstra: Implementations, experiments and data for a comparison of Dijkstra's algorithm and the SSSP algorithms of Duan et al.* (2026). GitHub. Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/anmoltutejagithub/bmssp-vs-dijkstra

[25] F. Li, D. Cheng, M. Hadjieleftheriou, G. Kollios, and S.-H. Teng, "On trip planning queries in spatial databases," in *Advances in Spatial and Temporal Databases (SSTD)*, Lecture Notes Comput. Sci., 2005, pp. 273–290, doi: 10.1007/11535331_16.

[26] M. Ansari, 2020, "Real dataset shortest path sample (road networks of the SSTD 2005 collection)," GitHub. [Online]. Available: https://github.com/mehdi-ansari/Real-dataset-shortest-path-sample

[27] T. Brinkhoff, "A framework for generating network-based moving objects," *GeoInformatica*, vol. 6, no. 2, pp. 153–180, Jun. 2002, doi: 10.1023/A:1015231126594.

[28] J. Leskovec and A. Krevl, Jun. 2014, "SNAP datasets: Stanford large network dataset collection," Stanford Univ. [Online]. Available: http://snap.stanford.edu/data

[29] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999, doi: 10.1126/science.286.5439.509.

[30] R. Bellman, "On a routing problem," *Quart. Appl. Math.*, vol. 16, no. 1, pp. 87–90, 1958.